Topic 7: Functions

- **Learning outcomes:**
  - (a) Explain the concept and advantages of functions in Python programming. (1 hour)

Table of Contents (TOC)

- Concept of functions
- Advantages of using functions
- Reusable feature

Definition

- **Function** is a reusable block of code that performs a specific task.

Concept of functions

- Python Functions are a block of statements that does a specific task
- The idea is to put some commonly or repeatedly done task together and make a function so that instead of writing the same code again and again for different inputs
- We can do the function calls to reuse code contained in it over and over again.

Concept of functions

Imagine a factory machine that makes drinks.

- Ingredients go in. (It can be orange, strawberry, chocolate)
- Machine follows instructions. (Machine processes the drinks)
- Finished product comes out. (Depending on what ingredient goes in)

The machine doesn’t care who uses it, what ingredient goes in — it just follows instructions every time.

Advantages of using functions

- Function makes your code **reusable**, **better organized**, and **more readable**.
- A function is a set of statements so they can be run more than once in a program.

Advantages of using functions: Reusable feature

Without function	User-defined function

a,b = 10,5
average = (a+b)/2
print("Average is: ", average)
 	
c,d =100,5
average = (c+d)/2
print("Average is: ", average)
 	
e,f =100,50
average = (e+f)/2
print("Average is: ", average)
 	
g,h = 9,50
average = (g+h)/2
print("Average is: ", average)
	def Average(a,b):
	    average = (a+b)/2;
	    print("Average is: ", average)
	
	Average(10,5)
	Average(100,5)
	Average(100,50)
	Average(9,50)

Advantages of using functions: Reusable feature

Calculate price after discount of different devices

```
def apply_discount(price):
    discount_rate = 0.10
    return price - (price * discount_rate)


# Different items
laptop_price = 3000
phone_price = 1500
headphone_price = 200

print("Laptop after discount:", apply_discount(laptop_price))
print("Phone after discount:", apply_discount(phone_price))
print("Headphone after discount:", apply_discount(headphone_price))
```

Advantages of using functions: Organise and readable

```
name = "Ali"
mark = 85
if mark >= 80:
    grade = "A"
elif mark >= 70:
    grade = "B"
elif mark >= 60:
    grade = "C"
elif mark >= 50:
    grade = "D"
else:
    grade = "F"

if grade == "F":
    status = "Fail"
else:
    status = "Pass"

print("Student Name :", name)
print("Marks        :", mark)
print("Grade        :", grade)
print("Status       :", status)
print("-----------------------------")
```

```
name = "Siti"
mark = 72
if mark >= 80:
    grade = "A"
elif mark >= 70:
    grade = "B"
elif mark >= 60:
    grade = "C"
elif mark >= 50:
    grade = "D"
else:
    grade = "F"

if grade == "F":
    status = "Fail"
else:
    status = "Pass"

print("Student Name :", name)
print("Marks        :", mark)
print("Grade        :", grade)
print("Status       :", status)
print("-----------------------------")
```

Without functions

Advantages of using functions: Organise and readable

```
def calculate_grade(mark):
    if mark >= 80:
        return "A"
    elif mark >= 70:
        return "B"
    elif mark >= 60:
        return "C"
    elif mark >= 50:
        return "D"
    else:
        return "F"


def check_pass_fail(grade):
    if grade == "F":
        return "Fail"
    else:
        return "Pass"
```

```
def print_result(name, mark):
    grade = calculate_grade(mark)
    status = check_pass_fail(grade)

    print("Student Name :", name)
    print("Marks        :", mark)
    print("Grade        :", grade)
    print("Status       :", status)
    print("-" * 30)


# Reusing the functions
print_result("Ali", 85)
print_result("Siti", 72)
print_result("John", 48)
print_result("Mei Ling", 66)
```

With functions

Exercise

1. State the definition of a function
2. Explain 2 advantages of a function

Summary

- Concept of functions
- Advantages of using functions
- Reusable feature

Topic 7: Functions

- **Learning outcomes:**
  - (b) Explain the differences between pre-defined functions and user-defined functions. (1 hour)

Table of Contents (TOC)

- Type of functions
- Definition of pre-defined function and user-defined functions
- Example of pre-defined functions
- Example of user-defined functions
- The characteristics of a user-defined functions
- How user-defined function works
- How to create user-defined function
- Exercise

Type of function	
• pre-defined functions	• User-defined functions

Definition of pre-defined functions	Definition of user-defined functions
• A function that are readily available for use	• A function that is defined by a programmer

TYPE OF FUNCTIONS

PRE-DEFINED FUNCTIONS

- Definition: A function that are **readily available** for use.
- Example :
  - `input()` – allowing user input
  - `print()` – prints to the standard output device
  - `pow()` – returns the value of `x` to the power of `y`
  - `max()` – returns the largest item in an iterable

USER-DEFINED FUNCTIONS

- Definition: A function that is **defined by a programmer**.
- Created by the user to perform a specific task or to solve problems.
- Example:

```
def my function():
  print("Hello from a function")

my_function()
```

Function Definition

Function Call

Example of pre-defined functions

```
print("Hello, World!")
```
← **print()** function to display String

```
input('Enter your name:')
```
← **input()** function allows user input

```
x = pow(4, 3)
```
← **pow( )** function returns the value of x to the power of y

```
a = (1, 2, 3, 4, 5)
x = sum(a)
```
← **sum()** function returns a value of the sum

Python has a set of pre-defined functions

For more information, visit https://www.w3schools.com/python/python_ref_functions.asp

Function	Description
abs()	Returns the absolute value of a number
all()	Returns True if all items in an iterable object are true
any()	Returns True if any item in an iterable object is true
ascii()	Returns a readable version of an object. Replaces none-ascii characters with escape character
bin()	Returns the binary version of a number
bool()	Returns the boolean value of the specified object
bytearray()	Returns an array of bytes
bytes()	Returns a bytes object
callable()	Returns True if the specified object is callable, otherwise False
chr()	Returns a character from the specified Unicode code.
classmethod()	Converts a method into a class method
compile()	Returns the specified source as an object, ready to be executed

Example of user-defined functions

Example 1	Example 2

`def message():`
`  print("Hello")`
 	
`message()`
	`def get_name(fname):`
	`  print(fname + " Refsnes")`
	
	`get_name("Emil")`
	`get_name("Tobias")`
	`get_name("Linus")`

The characteristics of a user-defined functions

- It groups a block of code together so that we can call it by name.
- It enables us to pass values into the the function when we call it.
- It can return a value (even if None).
- When a function is called, it has its own namespace (function space). Variables in the function are local to the function only (and disappear when the function exits).

```
def print_result(name, mark):
    grade = calculate_grade(mark)
    status = check_pass_fail(grade)

    print("Student Name :", name)
    print("Marks        :", mark)
    print("Grade        :", grade)
    print("Status       :", status)
    print("-" * 30)


# Reusing the functions
print_result("Ali", 85)
print_result("Siti", 72)
print_result("John", 48)
print_result("Mei Ling", 66)
```

How user-defined function works

Figure 1 shows

- when you **invoke/call** a function, Python remembers the place where it happened and *jumps* into the invoked function;
- the body of the function is then **executed**;
- reaching the end of the function forces Python to **return** to the place directly after the point of invocation.

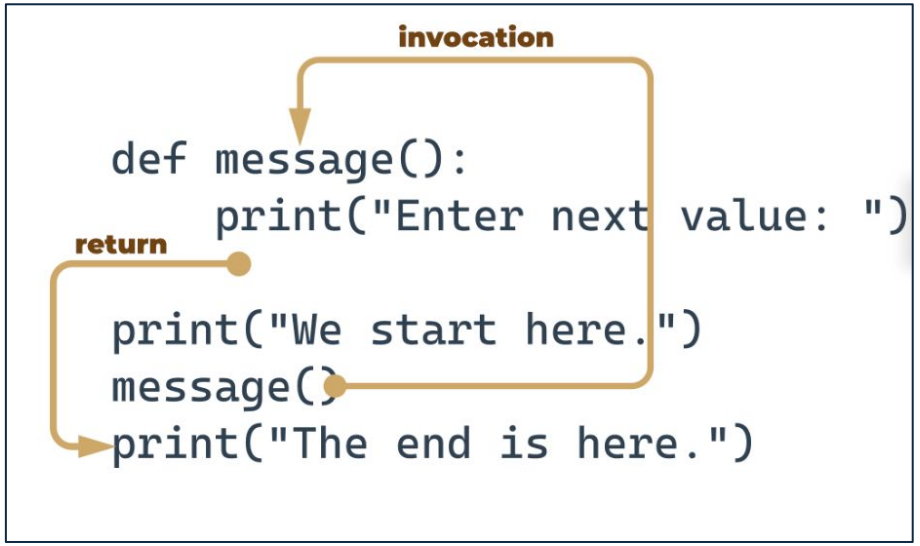


Figure 1

How user-defined function works

- Calling a function before declaring it is not possible.
- Python is an interpreted language, it does not allow invoking/ called a function before its definition.
- The order of function definitions matters, and you must define a function before you can call it.
- This is considered as a runtime error

How to create a user-defined function

Step 1 — **Create Function Definition**
- Function Header
- Function Body

Step 2 — **Write Function Call**

```
def message():
    print("Enter a value: ")

print("We start here.")
message()
print("We end here.")
```

Example of a program

Exercise

Question 1: The `input()` function is an example of a:

a) user-defined function

b) pre-defined function

Question 2: The following snippet is an example of a:

```
def message():
    print("Hello")

message()
```

a) user-defined function

b) pre-defined function

Exercise

Question 3: What happens when you try to invoke a function before you define it?

```
hi()

def hi():
    print("hi!")
```

Question 4: The following snippet is an example of a:

```
x = pow(4, 3)
print(x)
```

a) user-defined function

b) pre-defined function

Summary

- Type of functions
- Definition of pre-defined function and user-defined functions
- Example of pre-defined functions
- Example of user-defined functions
- The characteristics of a user-defined functions
- How user-defined function works
- How to create user-defined function
- Exercise

Topic 7: Functions

- **Learning outcomes:**
  - 7.3 Identify the use of common pre-defined Python functions such as print(), len(), input(), pow() and sqrt(). (1 hour)

Table of Contents (TOC)

- Recap definition of pre-defined function and user-defined functions
- Example of pre-defined functions
- Exercise

TYPE OF FUNCTIONS

PRE-DEFINED FUNCTIONS

- Definition: A function that are **readily available** for use.
- Example :
  - `input()` – allowing user input
  - `print()` – prints to the standard output device
  - `pow()` – returns the value of `x` to the power of `y`
  - `max()` – returns the largest item in an iterable

USER-DEFINED FUNCTIONS

- Definition: A function that is **defined by a programmer**.
- Created by the user to perform a specific task or to solve problems.
- Example:

```
def my function():
  print("Hello from a function")

my_function()
```

Function Definition

Function Call

Python has a set of pre-defined functions

For more information, visit https://www.w3schools.com/python/python_ref_functions.asp

Function	Description
abs()	Returns the absolute value of a number
all()	Returns True if all items in an iterable object are true
any()	Returns True if any item in an iterable object is true
ascii()	Returns a readable version of an object. Replaces none-ascii characters with escape character
bin()	Returns the binary version of a number
bool()	Returns the boolean value of the specified object
bytearray()	Returns an array of bytes
bytes()	Returns a bytes object
callable()	Returns True if the specified object is callable, otherwise False
chr()	Returns a character from the specified Unicode code.
classmethod()	Converts a method into a class method
compile()	Returns the specified source as an object, ready to be executed

print()

- **Purpose:**

Used to **display output** on the screen.

- **Example:**

```
print("Hello World")
print(100)
```

- **Use:**

- Display messages
- Show calculation results
- Debug programs

len()

- **Purpose:**

Used to **count the number of items** (in string or list).

- **Example:**

```
name = "Python"
print(len(name))       # Output: 6

numbers = [10, 20, 30]
print(len(numbers))    # Output: 3
```

- **Use:**

- Count characters in text
- Count items in a list
- Validate input length

input()

- **Purpose:**

Used to **get input from the user.**

⚠️ By default, input is stored as a string.

- **Example:**

```
name = input("Enter your name: ")
print(type(name))      #the data type will be str
print("Hello", name)
```

- **With numbers:**

```
age = int(input("Enter your age: "))
print(type(age))       #the data type will be int
print("Next year you will be", age + 1)
```

- **Use:**

- Collect user data
- Interactive programs
- Forms and calculators

pow()

- **Purpose:**

Used to calculate **power (exponents)**.

⚠️ Must import `math` module first.

- **Example:**

```
import math
print(math.pow(2, 3))   # 2³ = 8
print(math.pow(5, 2))   # 5² = 25
```

Equivalent to:

```
2 ** 3
```

- **Use:**

- Mathematical calculations
- Area formulas
- Scientific programs

sqrt()

- **Purpose:**

Used to calculate the **square root** of a number.

⚠️ Must import `math` module first.

- **Example:**

```
import math

print(math.sqrt(25))   # 5.0
print(math.sqrt(81))   # 9.0
```

- **Use:**

- Geometry calculations
- Distance formulas
- Engineering/science programs

Exercise

Question 1: The `input()` function is an example of a:

a) user-defined function

b) pre-defined function

Question 2: The following snippet is an example of a:

```
def message():
    print("Hello")

message()
```

a) user-defined function

b) pre-defined function

Exercise

Question 3: What happens when you try to invoke a function before you define it?

```
hi()

def hi():
    print("hi!")
```

Question 4: The following snippet is an example of a:

```
x = pow(4, 3)
print(x)
```

a) user-defined function

b) pre-defined function

Summary

- Type of functions
- Definition of pre-defined function and user-defined functions
- Example of pre-defined functions
- Example of user-defined functions
- The characteristics of a user-defined functions
- How user-defined function works
- How to create user-defined function
- Exercise

Topic 7: Functions

- **Learning outcomes:**

(d) Identify the components of a Python user-defined function, including function name, parameters, return statement and function body.

(1 hour)

Table of Contents (TOC)

- Introduction to Namespace
- Three components of a function
  i. Function Header
  ii. Function Body
  iii. Function Call
    A. Example Function Call
    B. How the program execute when the function is called
    C. Function Call without argument(s)
    D. Function Call with argument(s)
- Parameter vs argument

Introduction to Namespace

- Global space refers to the namespace that is outside of the function space.
- In this course we use the term global space and main space interchangeably.
- Moving forward, the main space is the term that we will use.

```
## Function space ##
def hi():
    print("hi!")
```

```
## Main space ##
hi()
```

LECTURE NOTES

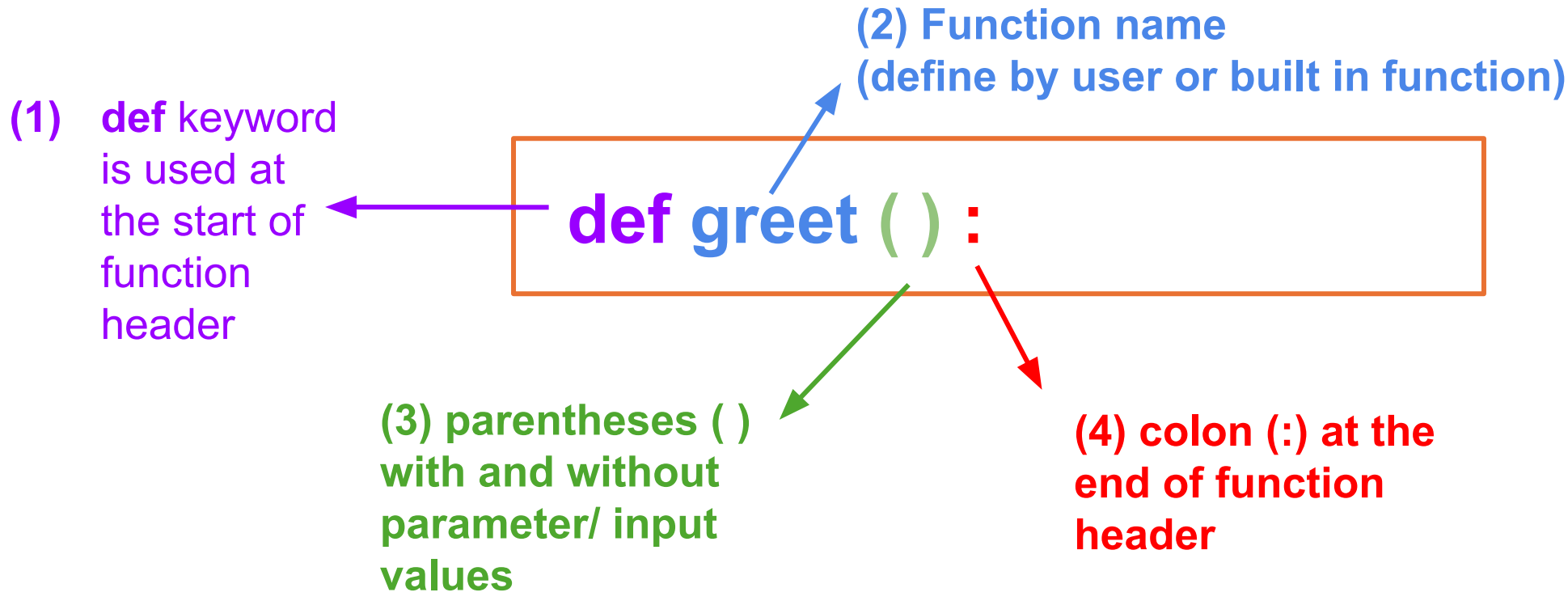
Three components of a function:

i. Function Header
ii. Function Body
iii. Function Call



i. Function Header

Four Elements in Function Header



Examples of Function Header

Function header without parameter

```
def status():

def message():
```

Function header with parameter

```
def find_total(a, b):

def display_lect(name):

def find_avg(no1, no2, no3):
```

- The function header name is using snake_case naming convention
- Parameter - Input value(s) for the function to process.
- Example - no1, no2 and no3 that will be processed to find the average

Exercise 1

Exercise 1.1

(i) Create a function header named `determine_grade` without parameter.

```
def __________ _
```

(ii) Create a function header named `display_message` without parameter.

```
_______ _______ () :
```

Exercise 1.1

(iii) Create a function header named `calc_total` that accepts two parameters, `a` and `b`.

```
_______ total(___, b):
```

(iv) Create a function header named `calc_sum` that accepts two parameters named `num1` and `num2`.

```
___ _________ (_____, _________):
```

Exercise 1.2

(i) This function named `display_msg()` is to display "Welcome back". Identify the **errors** from this snippet below.

```
## create function ##

def display_msg();
   print("Welcome back")

## main ##
msg()
```

Exercise 1.2

(i) This function named `display_msg()` is to display "Welcome back". Identify the **errors** from this snippet below.

```
  File "/tmp/ipython-input-2761624640.py",
    def display_msg();
                     ^
SyntaxError: expected ':'
```

```
NameError                                 Traceback
/tmp/ipython-input-1476903710.py in <cell line: 0>()
      5
      6 ## main ##
----> 7 msg()
      8
      9

NameError: name 'msg' is not defined
```

Exercise 1.2

(ii) This function named `display_msg()` is to display "Welcome back". Identify the error from this snippet below.

```
## create function ##

def display_msg():
print("Welcome back")

## main ##
display_msg()
```

Exercise 1.2

(iii) This function named `display_msg()` is to display "Welcome back". Identify the error from this snippet below.

```
## create function ##

def display_msg:
   print("Welcome back")

## main ##
display_msg()
```

Exercise 1.2

(iv) This function named `display_msg()` is to display "Welcome back". Identify the error from this snippet below.

```
## create function ##

defdisplay_msg():
   print("Welcome back")

## main ##
display_msg()
```

ii. Function Body

- The body of function consists of **one or more valid statements**.(Eg: to calculate length, to print value)

- Each statement must be indented with the **same indentation** to form a block.

ii) Function Body (without or with return values)

a) Function body without return value
(no **return** keyword)

b) Function body with return value
(must have **return** keyword)

Example of Function Body

(a) Function body without return value (no return keyword)

Function body

```
def avg():
    total = 100 + 20 + 30
    average = total / 3
    print (average)
```

```
def calcAvg(num1,num2,num3):
    total = num1 + num2 + num3
    avg = total/3
    print(total)
```

Function body

(b) Function body with return value (must have return keyword)

```
def calcAvg(num1,num2,num3):
    total = num1 + num2 + num3
    avg = total/3
    return avg
```

Function body with return keyword

```
def greet(message):
    return message
```

Function body with return keyword

iii) A. Function Call

- Function without function call is meaningless.
- Therefore, we need to create function call.
- To call a function, use the **function name** followed by parenthesis(), with or without arguments.

iii) A. Function Call - continued

- Once the function is invoked (called) in main program space, the parameters inside the parentheses () is called **arguments**.
- The number of arguments **must match with** parameters of the function.
- Arguments must have values

```
## create function to add two numbers ##
def add_numbers(a, b):
    result = a + b
    return result

## main ##
sum_result = add_numbers(5, 3)
print("The sum is:", sum_result)
```

Parameters

Function call

Arguments

Example Function Call Without Arguments

function space

```
def greet():
    print('Hello World!')
```

function header

function body

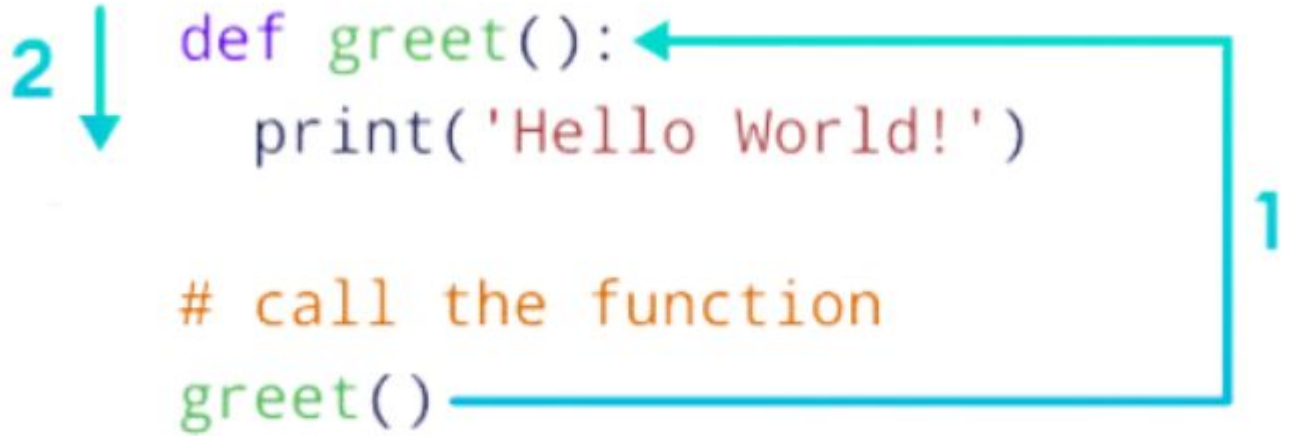
main program space

```
# call the function
greet()
```

function call (name must be the same as function name)

LECTURE NOTES

iii) B. How the program execute when the function is called



1. When the function `greet()` is called from the main space, the program's control transfers to the function definition in function space.
2. All the code inside the function is executed.

iii) C. Function Call without argument(s)

1. Function call without argument(s)
    - 1.1 Without return value
    - 1.2 With return value

Return value is the **result** that a function sends back to the main space after it finishes running.

1. Function Call without argument(s)

1.1 Without return value

```
## create function ##
def greet():
 print ('Hi there')

## main ##
greet()
```

function call

Output:

```
Hi there
```

1. Function Call without argument(s)

1.2 With return value

```
## create function ##
def get_greeting():
    return "Hello, welcome back!"

## main ##
print(get_greeting())
```

*Direct print from the returned value

```
## create function ##
def get_greeting():
    return "Hello, welcome back!"

## main ##
message = get_greeting()
print(message)
```

*Store returned value in a variable to be used later

Output:

```
Hello, welcome back!
```

iii) D. Function Call with argument(s)

1. Function call with argument(s)
    1.1 Without return value
    1.2 With return value

iii) D. Function Call with argument(s)

2.1 Without return value

```
## create funtion to print a grade ##
def print_grade(grade):
    print("The grade is:", grade)

## main ##
print_grade("A")
```

function call with argument

Output:

```
The grade is: A
```

2.2 With return value

```
## create function to add two numbers ##
def add_numbers(a, b):
    result = a + b
    return result

## main ##
sum_result = add_numbers(5, 3)
print("The sum is:", sum_result)
```

function call with arguments

Output:

```
The sum is: 8
```

Parameter vs Argument

- Parameter: in function header (function space)
- Argument: in function call (main program space)

Recap

LECTURE NOTES


```
## create function calcArea ##
def calcArea(length, width):
    area_rectangle = length * width
    return area_rectangle

## main ##
print(calcArea(2,3))
print(calcArea(4,20))
```

parameters (variable) — length, width

arguments (values) — 2, 3 / 4, 20

Output:

```
6
80
```

LECTURE NOTES


```
## create function calcArea ##
def calcArea(length, width):
    area_rectangle = length * width
    return area_rectangle


## main ##
print(calcArea(2,3))
print(calcArea(4,20))
```

parameters used in function header

arguments used in function call

LECTURE NOTES

```
## create function calcArea ##
def calcArea(length, width):
    area_rectangle = length * width
    return area_rectangle

## main ##
areaRect = calcArea(2,3)
print (areaRect)
areaRect = calcArea(4,20)
print (areaRect)
```

we could assign function call to a variable

Output:

```
6
80
```

Exercise 1

Write a function to calculate area of a rectangle (with and without return value)

```
length = float(input("Enter the length of the rectangle: "))
width = float(input("Enter the width of the rectangle: "))
area = length * width
print("The area of the rectangle is:", area)
```

Exercise 2

Exercise 2.1

(i) Create a function header named `bmi()` that accepts two parameters named `height` and `weight`.

(ii) Call the previous function by using the values 1.53 and 55 for height and weight respectively. (Assume there is a return values)

Exercise 2.2

(i) Create a function header named `sum()` that accepts three integers value named `num1`, `num2` and `num3`.

(ii) Call the previous function by using the values 10, 20 and 30 for num1, num2, and num3 respectively. (Assume there is a return values)

(iii) Identify the arguments and parameters from (i) and (ii)

Summary

- Introduction to Namespace
- Three components of a function
  i. Function Header
  ii. Function Body
  iii. Function Call
    A. Example Function Call
    B. How the program execute when the function is called
    C. Function Call without argument(s)
    D. Function Call with argument(s)
- Parameter vs argument